

Fractional Knapsack & the Greedy Strategy

Understanding the fractional knapsack problem and its greedy-based solution gives you a blueprint for **allocating any limited, continuous resource** in backend systems or data pipelines.

Concept Overview

You have:

- A knapsack (capacity C)
- n items, each with
 - a value v_i
 - a weight w_i
- You may take **any fraction** of an item (e.g. half of an item)

Goal: maximize total value without exceeding capacity.

Greedy Solution in Simple Steps

1. Compute **value density** for each item: $\text{density}_i = v_i / w_i$
2. Sort items by density_i in **descending** order.
3. Initialize

Js Copy

```
let capacityLeft = C;
let totalValue = 0;
```

4. For each item in sorted list:
 - If $\text{capacityLeft} \geq w_i$, take the whole item:

Js Copy

```
totalValue += v_i;
capacityLeft -= w_i;
```

- Else, take only a fraction $f = \text{capacityLeft} / w_i$:

Js Copy

```
totalValue += v_i * f;
capacityLeft = 0;
break; // no more capacity
```

This guarantees the highest-possible total value because you always “buy” the richest value-per-unit-weight first.

Real-World Reflection in Backend & Data Engineering

At its core, fractional knapsack models **how to divvy up a scarce, divisible resource**. In many backend or data-engineering scenarios, you face exactly this pattern:

- **Bandwidth Allocation** You have a fixed network pipe. Different data streams yield different “importance per megabyte.” Allocate bandwidth slices to the streams
- **CPU or Container Shares** A cluster has limited CPU cycles. Microservices vary in business value per CPU-second. Assign fractional CPU shares to maximize total business value.
- **Cache / Memory Partitioning** Your cache is finite. Query groups have different cache-hit rates (value) per megabyte (cost). Carve up cache space fractionally according to hitrate/MB.
- **Cloud Budgeting for Pipelines** You’ve got a monthly cloud spend limit. ETL jobs deliver data-quality improvements at different dollars-per-unit. Split your budget fractionally among jobs to maximize data quality.

In every case you:

1. Define a **value** metric (importance, throughput, quality gain).
2. Define a **cost** metric (bandwidth, CPU time, memory, dollars).
3. Compute **value-per-cost**, sort descending, and “fill” until your resource runs out.

Example Scenario: API Rate Limits

Suppose you can make 1,000 API calls per minute across three external services:

- API A: Value = 100 pts/call, Cost = 0.10 s/call
- API B: Value = 40 pts/call, Cost = 0.05 s/call
- API C: Value = 80 pts/call, Cost = 0.20 s/call

Compute quality-per-second:

- A: $100 \div 0.10 = 1,000$ pts/s
- B: $40 \div 0.05 = 800$ pts/s
- C: $80 \div 0.20 = 400$ pts/s

Greedy fill:

1. Use as many calls to A as you can.
2. Then B.
3. Then C (if time remains).

This maximizes total quality in your fixed 60 s window—exactly the fractional knapsack logic.

Key Takeaway

Whenever you must **divide a continuous, limited resource** across competing consumers to maximize a benefit, think fractional knapsack. Its greedy strategy is simple fast, and directly applicable to backend resource scheduling, data-engineering pipelines, and cloud cost optimizations.

